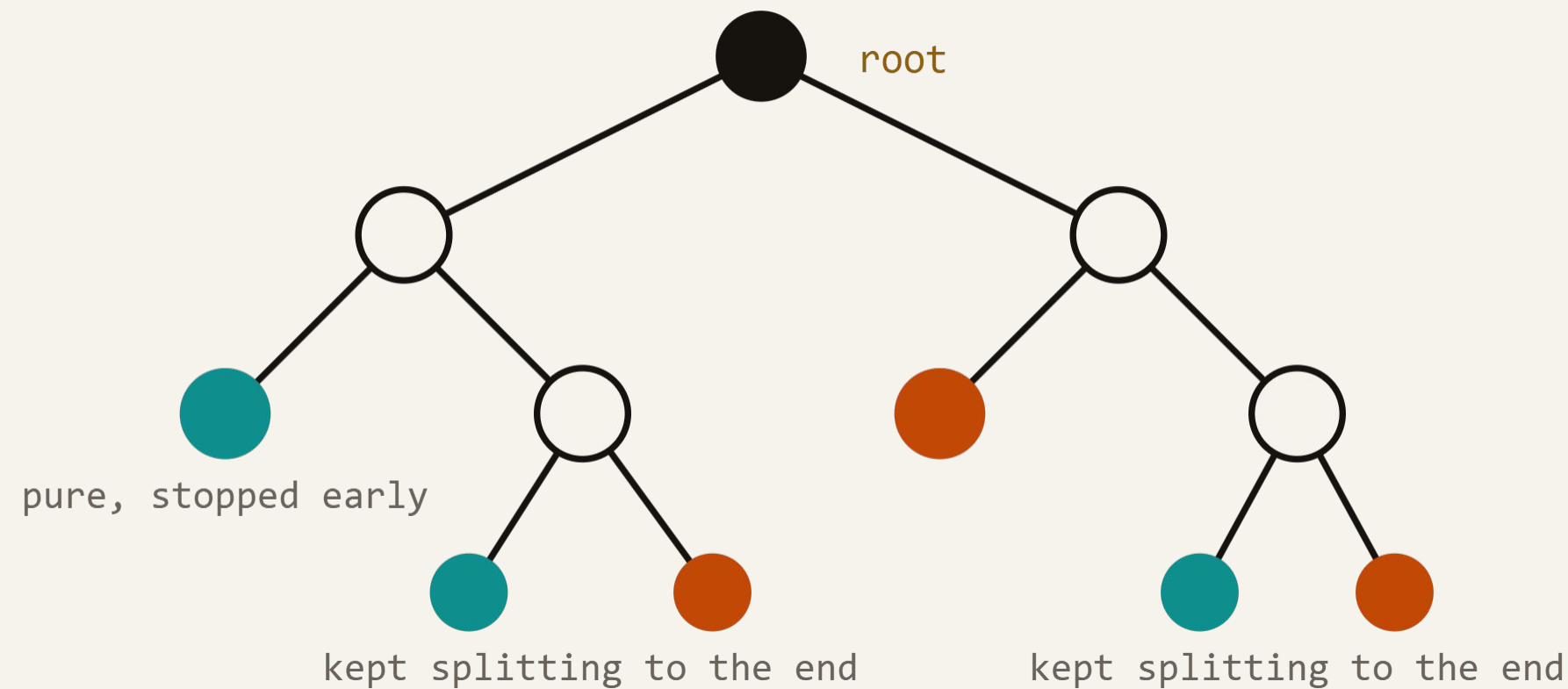


TWO WAYS TO STOP A TREE FROM GROWING TOO FAR

Pre-Pruning and Post-Pruning

Left alone, a decision tree splits until every leaf is pure. It scores perfectly on the training data and poorly on everything else. Pruning is how you stop it.

Nothing Stops a Tree by Default



The stopping rule in the default settings is simply "**stop when the leaf is pure**".
`max_depth=None`.

Some branches reach a pure node quickly and stop there. Others keep carving off two or three rows at a time.

The tree grows as deep as it needs to explain **every single training row**. That is not what we wanted.

■ THE COST

Perfect on the training set. Poor on everything else.

SYMPTOM

train accuracy ↑↑

test accuracy ↓↓

A tree that kept splitting until every leaf was pure has a rule for every training row by construction.

IN BIAS AND VARIANCE

Low bias · error on training data

High variance · error on unseen data

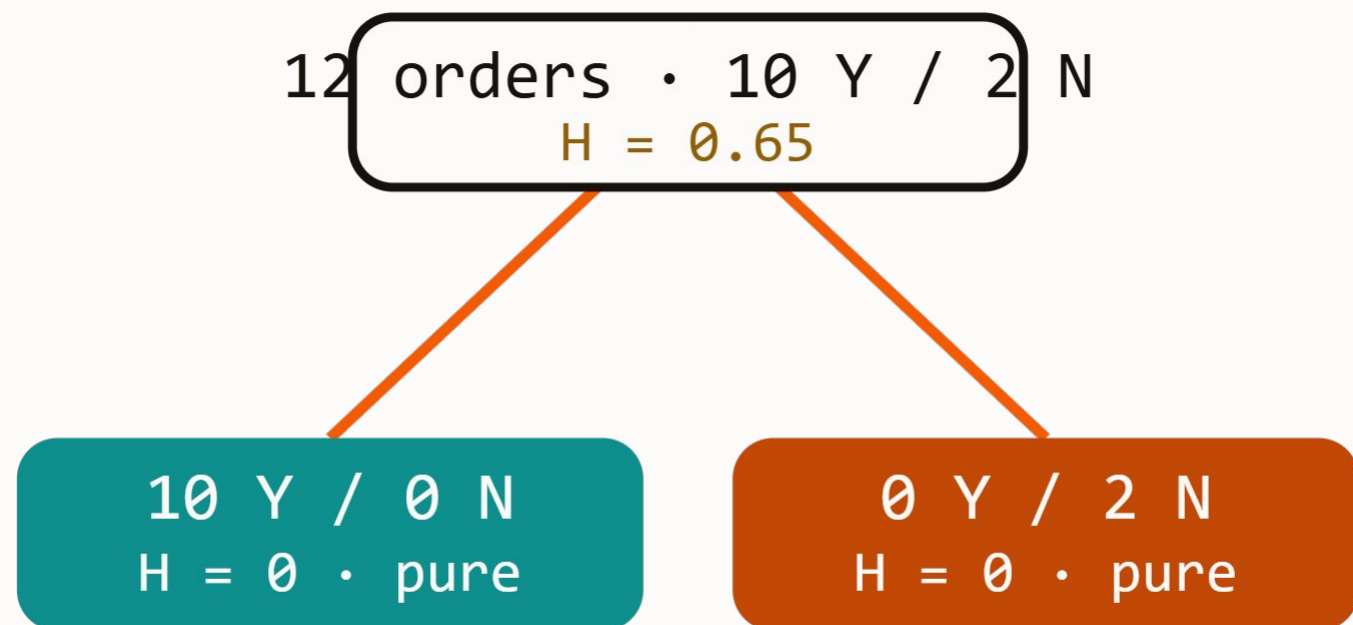
The same overfitting signature we saw in regression before regularisation.

PLAINLY

The tree memorised the training set instead of learning the pattern in it.

A model that only works on rows it has already seen is of no use to anyone.

A Split That Exists to Catch Two Rows



information gain = 0.65, the whole of it

By the maths this is a **great** split. Both children are pure, the gain is maximal, and the algorithm will always take it.

But the parent already said something useful: 10 of 12 on time. Predict on time there and you are right 83% of the time.

The split exists only to catch **two rows**. They may be a real pattern, or one rainy Tuesday and one wrong address. The tree cannot tell, and builds a rule either way.

Pruning: Keep the Tree Smaller On Purpose

POST-PRUNING

Grow it fully, then cut back

Build the whole tree first. Then remove the branches that were only memorising, and turn those nodes into leaves.

PRE-PRUNING

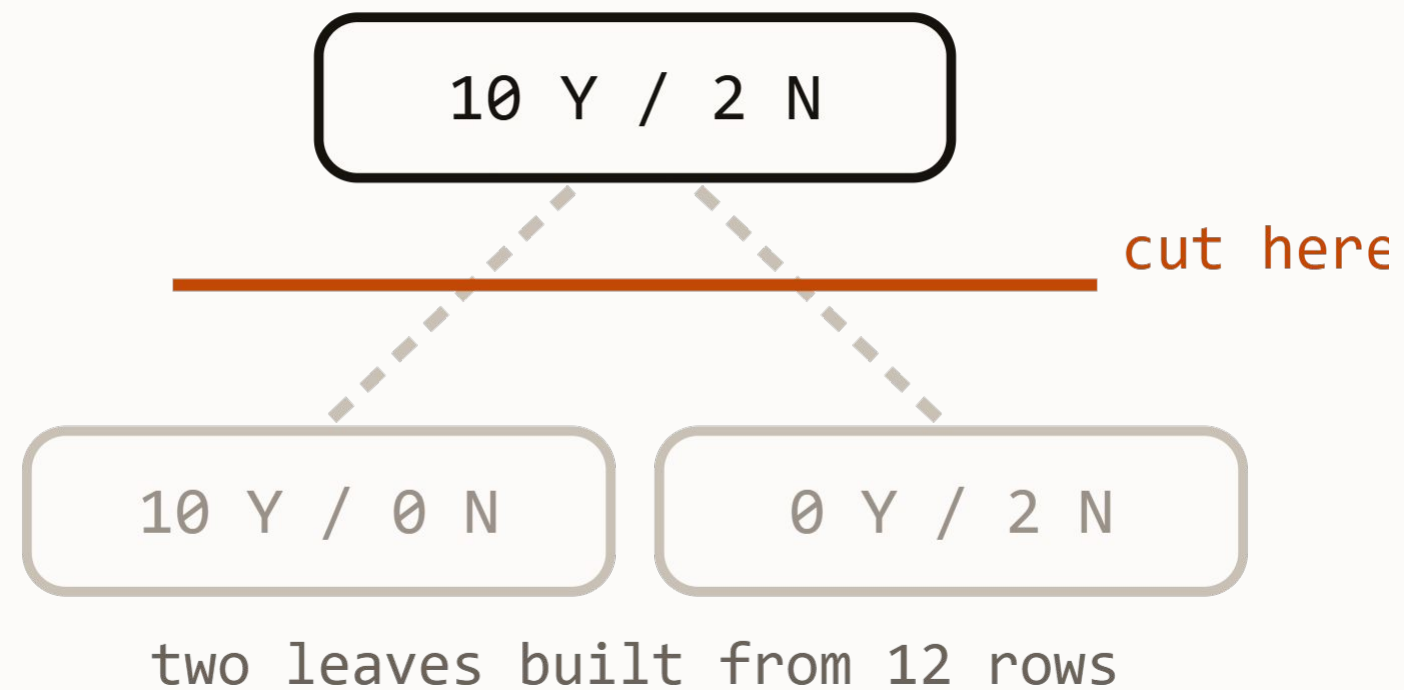
Set limits before you start

Fix the depth and the minimum node sizes up front, so the greedy branches never get built at all.

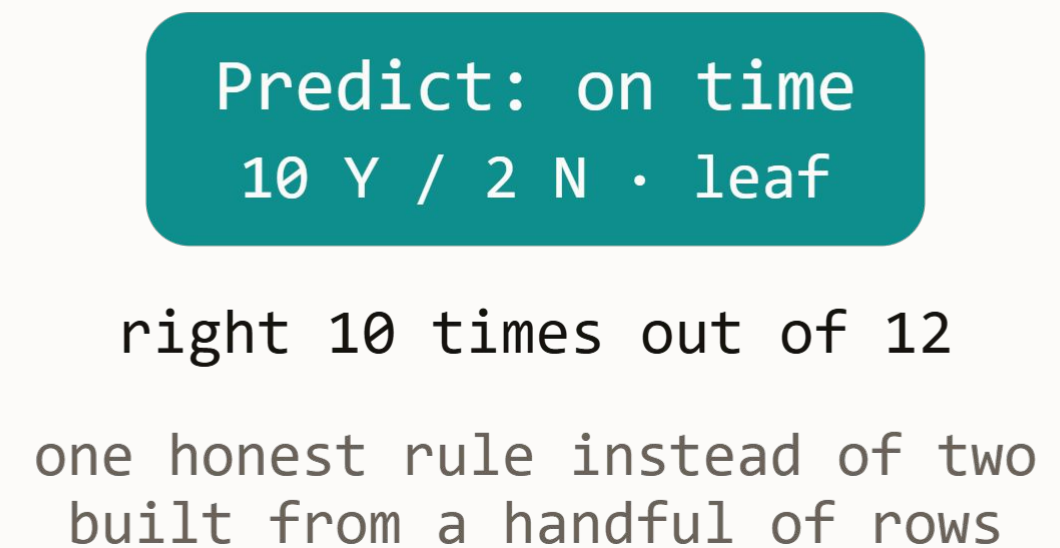
Both trade a little training accuracy for real test accuracy. The only difference is **when** you do the cutting.

Cut the Branch, Keep the Majority

BEFORE · FULLY GROWN



AFTER · PRUNED



Training accuracy drops by those 2 rows. In exchange, every new order that lands here gets the sensible answer instead of a rule built from two examples.

Build First, Cut Second

01

Construct the full tree

Let it split all the way down, exactly as the default settings do.

02

Prune it back by depth

Cut the branches that do not earn their place and turn those nodes into majority leaves.

03

Best on smaller datasets

Step 01 pays the full cost of a complete tree before anything is cut. On lakhs of rows that is expensive, and part of it is thrown away.

In scikit-learn this is `ccp_alpha`, cost complexity pruning, applied after the tree is fitted.

PRE-PRUNING

Set the Limits Before You Fit

```
max_depth = 3
```

The tree may not go deeper than 3 levels, whatever the gain says.

```
min_samples_split = 20
```

Do not even consider splitting a node holding fewer than 20 rows.

```
min_samples_leaf = 10
```

Never create a leaf smaller than 10 rows. This alone would have blocked our 2-row leaf.

```
max_features, criterion, splitter
```

How many features each node may consider, gini or entropy, best or random split.

The full tree is **never built**. The limits are in place while it is being constructed, so the greedy branches never appear.

And you do not guess these numbers. You tune them, with **GridSearchCV**, exactly as you tuned C and gamma for SVM.

Pre-pruning is hyperparameter tuning while the tree is being built. Nothing more mysterious than that.

Which One, and When

POST-PRUNING

Order: build the full tree, then cut.

Strength: the cut is decided with the whole tree visible, so you cut what actually failed to help.

Cost: you pay for a complete tree first, then throw part of it away.

Suits smaller datasets.

PRE-PRUNING

COMMON

Order: set the limits, then build once within them.

Strength: cheap, because the deep branches never get built at all.

Cost: the limits are fixed blind, so a useful branch can be blocked along with the wasteful ones.

Fits straight into the GridSearchCV you are already running.

Losing Accuracy On Purpose?

"Pruning lowers training accuracy. How is that better?"

Training accuracy was never the goal. The model exists to answer questions about orders it has not seen.

full tree · train 100% · test 70% pruned ·
train 92% · test 85%

Pruning trades the number you do not care about for the one you do.

"How do I know my tree is overfitting?"

Print both accuracies. If training sits far above test, the tree is too deep. That gap is your whole diagnostic.

Then reduce `max_depth` or raise `min_samples_leaf` and watch the gap close.

A tree with a depth of 30 on a few thousand rows is almost always overfitting, whatever the training score says.

What to Remember

Unstopped, a tree memorises. It splits until every leaf is pure: low bias, high variance, train accuracy high and test accuracy poor.

Post-pruning: build fully, then cut back. Good on smaller datasets. In scikit-learn, `ccp_alpha`.

Pre-pruning: limits first, built once. `max_depth`, `min_samples_leaf`, tuned with `GridSearchCV`.

The trade is deliberate. Give up a little training accuracy, get back test accuracy. Judge on the test set only.